

CHAPTER – 4

PROJECT – 1

SCHOOL STUDENT MANAGEMENT SYSTEM

Objective

The objective of this project is to develop a simple School Student Management System using Object-Oriented Programming (OOP) concepts in Python. The project focuses on representing student information through classes, displaying student details, and updating student grades efficiently using class methods. It demonstrates the practical implementation of encapsulation, object creation, and method invocation in Python.

Technologies Used

- Python 3: Used as the programming language to implement the complete student management system.
- Object-Oriented Programming (OOP): Utilized to organize the program using classes, objects, constructors, and methods.
- Visual Studio Code: Used as the development environment for writing, executing, and debugging the Python program
- Python Terminal: Used to execute the program and display the output.

Workflow Explanation

- Create Student Class: Define a Student class with a constructor (`__init__`) to initialize student ID, name, age, and grade.
- Initialize Student Objects: Create student objects by passing the required information to the constructor.
- Display Student Information: Implement the `display_details()` method to print each student's ID, name, age, and grade in a structured format.
- Update Student Grade: Implement the `update_grade()` method to modify the student's grade and confirm the successful update.
- Display Updated Information: After updating the grade, display the student details again to verify that the changes have been applied successfully.

Program :**PART 1: Constructor Initialization**

```

2 class Student:
3     def __init__(self, student_id, name, age, grade):
4         self.student_id = student_id
5         self.name = name
6         self.age = age
7         self.grade = grade

```

This part implements the `__init__()` constructor, which initializes the student object's attributes such as student ID, name, age, and grade whenever a new object is created.

PART 2: Student Methods Definition

```

9     def display_details(self):
10         print("Student ID:", self.student_id)
11         print("Name:", self.name)
12         print("Age:", self.age)
13         print("Grade:", self.grade)
14
15     def update_grade(self, new_grade):
16         self.grade = new_grade
17         print() #blank line
18         print("Grade updated successfully!")

```

This part defines the member methods of the Student class. The `display_details()` method displays the student's information, while the `update_grade()` method updates the student's grade and confirms the successful modification.

PART 3: Student Object Creation & Initial Display

```

20 student1 = Student(101, "Rahul", 15, "A")
21
22 print("\nBefore Grade Update:")
23 print() #blank line
24 student1.display_details()

```

This part creates a Student object by passing the required details to the constructor and displays the student's information before updating the grade. It demonstrates object creation and method invocation.

PART 4: Grade Update & Updated Display

```

26 student1.update_grade("A+")
27
28 print("\nAfter Grade Update:")
29 print() #blank line
30 student1.display_details()

```

This part updates the student's grade using the `update_grade()` method and displays the modified student information to verify that the grade has been successfully updated.

Output:

```
Before Grade Update:

Student ID: 101
Name: Rahul
Age: 15
Grade: A

Grade updated successfully!

After Grade Update:

Student ID: 101
Name: Rahul
Age: 15
Grade: A+
```

Conclusion

This project successfully demonstrates the implementation of Object-Oriented Programming principles using Python. By organizing student information within a class and performing operations through methods, the system provides a clean and modular approach to managing student records. It also improves code readability, maintainability, and reusability.

Key Takeaways

- Method Implementation: Understood how member functions perform operations such as displaying and updating object data.
- Encapsulation: Organized related data and functions inside a single class, improving code structure and maintainability.
- Code Reusability: Learned how objects created from the same class can manage multiple student records without duplicating code. Object-Oriented Design: Learned how to create classes and objects to represent real-world entities efficiently.
- Constructor Usage: Gained practical experience using constructors (`__init__`) to initialize object attributes automatically